

Fault-Based Testing

Learning objectives

- Understand the basic ideas of fault-based testing
 - How knowledge of a fault model can be used to create useful tests and judge the quality of test cases
 - Understand the rationale of fault-based testing well enough to distinguish between valid and invalid uses
- Understand mutation testing as one application of fault-based testing principles

Let's count marbles ... a lot of marbles

- Suppose we have a big bowl of marbles. How can we estimate how many?



Photo credit: (c) KaCey97007 on Flickr,
Creative Commons license

- I don't want to count every marble individually
- I have a bag of 100 other marbles of the same size, but a different color
- What if I mix them?

Estimating marbles



- I mix 100 black marbles into the bowl
 - Stir well ...
- I draw out 100 marbles at random
- 20 of them are black
- How many marbles were in the bowl to begin with?

Estimating Test Suite Quality

- Now, instead of a bowl of marbles, I have a program with bugs
- I add 100 new bugs
 - Assume they are exactly like real bugs in every way
 - I make 100 copies of my program, each with one of my 100 new bugs
- I run my test suite on the programs with seeded bugs ...
 - ... and the tests reveal 20 of the bugs
 - (the other 80 program copies do not fail)
- What can I infer about my test suite?

Basic Assumptions

- We'd like to judge effectiveness of a test suite in finding real faults, by measuring how well it finds seeded fake faults.
- Valid to the extent that the seeded bugs are representative of real bugs
 - Not necessarily identical (e.g., black marbles are not identical to clear marbles); but the differences should not affect the selection
 - E.g., if I mix metal ball bearings into the marbles, and pull them out with a magnet, I don't learn anything about how many marbles were in the bowl

Mutation testing

- A *mutant* is a copy of a program with a *mutation*
- A *mutation* is a syntactic change (a seeded bug)
 - Example: change $(i < 0)$ to $(i \leq 0)$
- Run test suite on all the mutant programs
- A mutant is *killed* if it fails on at least one test case
- If many mutants are killed, infer that the test suite is also effective at finding real bugs

Consider the code:

```
if(boolean1&&boolean2){  
    int a = 5;  
}else{  
    int a = 0;  
}
```

```
if(boolean1&&boolean2){  
    int a = 5;  
}else{  
    int a = 0;  
}
```

- Mutation testing works one at a time
- Common terms “mutated”:
 - Boolean: >, <, ==, <=, =
 - Arithmetic: +, -, *, /
 - Replacement of same-scope variables
 - Replace true/false
 - Specific deletion of a statement

What happens if you change the && to ||?

```
if(boolean1 || boolean2){  
    int a = 5;  
}else{  
    int a = 0;  
}
```

- Input:
 - boolean1==1
 - boolean2==0
- Condition 1: Difference between original and mutant?
 - Yes
- Therefore the mutant is said to be “killed” and the test is successful

- **Open Source Testing**

- Jester (for Java)
- Pester (for Python)
- Nester (for C languages)
- μ Java (MuJava)

- Joint project between Korea Advanced Institute of Science and Technology (KAIST) in S. Korea and George Mason University

- **Commercial Testing**

- Insure++
 - Created/Maintained by Parasoft, Inc.
 - Version 4.0



- The “Mothra Project”

- 1980-Current study/development of Mutation Tests
- <http://cs.gmu.edu/~offutt/rsrch/mut.html>

- Parasoft Insure++

- <http://www.parasoft.com/jsp/home.jsp>

- MuJava

- <http://cs.gmu.edu/~offutt/mujava/>

What do I need to believe?

- Mutation testing uses seeded faults (syntactic mutations) as black marbles
- Does it make sense? What must I assume?
 - What must be true of black marbles, if they are to be useful in counting a bowl of pink and red marbles?

Mutation testing assumptions

- Competent programmer hypothesis:
 - Programs are nearly correct
 - Real faults are small variations from the correct program
 - => Mutants are reasonable models of real buggy programs
- Coupling effect hypothesis:
 - Tests that find simple faults also find more complex faults
 - Even if mutants are not perfect representatives of real faults, a test suite that kills mutants is good at finding real faults too

Mutation Operators

- Syntactic change from legal program to legal program
 - So: Specific to each programming language. C++ mutations don't work for Java, Java mutations don't work for Python
- Examples:
 - crp: constant for constant replacement
 - for instance: from $(x < 5)$ to $(x < 12)$
 - select from constants found somewhere in program text
 - ror: relational operator replacement
 - for instance: from $(x \leq 5)$ to $(x < 5)$
 - vie: variable initialization elimination
 - change `int x =5;` to `int x;`

Live Mutants

- Scenario:
 - We create 100 mutants from our program
 - We run our test suite on all 100 mutants, plus the original program
 - The original program passes all tests
 - 94 mutant programs are killed (fail at least one test)
 - 6 mutants remain *alive*
- What can we learn from the living mutants?

How mutants survive

- A mutant may be equivalent to the original program
 - Maybe changing $(x < 0)$ to $(x \leq 0)$ didn't change the output at all! The seeded "fault" is not really a "fault".
 - Determining whether a mutant is equivalent may be easy or hard; in the worst case it is undecidable
- Or the test suite could be inadequate
 - If the mutant could have been killed, but was not, it indicates a weakness in the test suite
 - But adding a test case for just this mutant is a bad idea. We care about the real bugs, not the fakes!

Variations on Mutation

- Weak mutation
- Statistical mutation

Weak mutation

- Problem: There are lots of mutants. Running each test case to completion on every mutant is expensive
 - Number of mutants grows with the square of program size
- Approach:
 - Execute meta-mutant (with many seeded faults) together with original program
 - Mark a seeded fault as “killed” as soon as a difference in intermediate state is found
 - Without waiting for program completion
 - Restart with new mutant selection after each “kill”

Statistical Mutation

- Problem: There are lots of mutants. Running each test case on every mutant is expensive
 - It's just too expensive to create N^2 mutants for a program of N lines (even if we don't run each test case separately to completion)
- Approach: Just create a random sample of mutants
 - May be just as good for *assessing* a test suite
 - Provided we don't design test cases to kill particular mutants (which would be like selectively picking out black marbles anyway)

In real life ...

- Fault-based testing is a widely used in semiconductor manufacturing
 - With good *fault models* of typical manufacturing faults, e.g., “stuck-at-one” for a transistor
 - But fault-based testing for *design errors* is more challenging (as in software)
- Mutation testing is not widely used in industry
 - But plays a role in software testing research, to compare effectiveness of testing techniques
- Some use of fault models to design test cases is important and widely practiced

Integration Testing

Testing connections among separate program units

- In Java, testing the way classes, packages and components are connected
 - “*Component*” is used as a generic term
- This tests features that are unique to object-oriented programming languages
 - inheritance, polymorphism and dynamic binding
- Integration testing is often based on couplings – the explicit and implicit relationships among software components

Integration Mutation

- Faults related to component integration often depend on a **mismatch of assumptions**
 - Callee thought a list was sorted, caller did not
 - Callee thought all fields were initialized, caller only initialized some of the fields
 - Caller sent values in kilometers, callee thought they were miles
- Integration mutation focuses on **mutating the connections** between components
 - Sometimes called “*interface mutation*”
 - Both caller and callee methods are considered

Four Types of Mutation Operators

- Change a calling method by modifying values that are sent to a called method
- Change a calling method by modifying the call
- Change a called method by modifying values that enter and leave a method
 - Includes parameters as well as variables from higher scopes (class level, package, public, etc.)
- Change a called method by modifying return statements from the method

Five Integration Mutation Operators

1. IPVR — Integration Parameter Variable Replacement

Each parameter in a method call is replaced by each other variable in the scope of the method call that is of compatible type.

- **This operator replaces primitive type variables as well as objects.**

2. IUOI — Integration Unary Operator Insertion

Each expression in a method call is modified by inserting all possible unary operators in front and behind it.

- **The unary operators vary by language and type**

3. IPEX — Integration Parameter Exchange

Each parameter in a method call is exchanged with each parameter of compatible types in that method call.

- **max (a, b) is mutated to max (b, a)**

Five Integration Mutation Operators (2)

4. IMCD — Integration Method Call Deletion

Each method call is deleted. If the method returns a value and it is used in an expression, the method call is replaced with an appropriate constant value.

- **Method calls that return objects are replaced with calls to “new ()”**

5. IREM — Integration Return Expression Modification

Each expression in each return statement in a method is modified by applying the UOI operators.

Integration Mutation Operators—Example

1. *IPVR – Integration Parameter Variable Replacement*

```
MyObject a, b;  
...  
callMethod (a);  
Δ callMethod (b);
```

2. *IUOI – Integration Unary Operator Insertion*

```
callMethod (a);  
Δ callMethod  
(a++);
```

Integration Mutation Operators—Example

3. IPEX – Integration Parameter Exchange

```
Max (a, b);  
Δ Max (b, a);
```

4. IMCD – Integration Method Call Deletion

```
X = Max (a, b);  
Δ X = new Integer (3);
```

5. IREM – Integration Return Expression Modification

```
int myMethod ()  
{  
    return a;  
Δ return ++a;  
}
```

Integration Mutation Operators—Example

1. *IPVR – Integration Parameter Variable Replacement*

```
MyObject a, b;  
...  
callMethod (a);  
Δ callMethod (b);
```

2. *IUOI – Integration Unary Operator Insertion*

```
callMethod (a);  
Δ callMethod  
(a++);
```

Object-Oriented Mutation

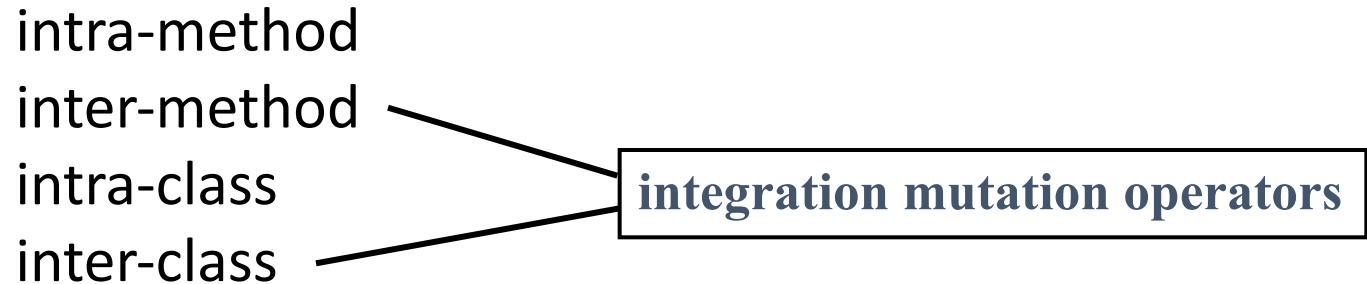
Testing Levels

intra-method

inter-method

intra-class

inter-class



integration mutation operators

- These five operators can be applied to non-OO languages
 - C, Pascal, Ada, Fortran, ...
- They do not support object oriented features
 - Inheritance, polymorphism, dynamic binding
- Two other language features that are often lumped with OO features are information hiding (encapsulation) and overloading
- Even experienced programmers often get encapsulation and access control wrong

Encapsulation, Information Hiding and Access Control

- Encapsulation : An abstraction mechanism to implement information hiding, which is a design technique that attempts to protect parts of the design from parts of the implementation
 - Objects can restrict access to their member variables and methods
- Java provides four **access levels** (C++ & C# are similar)
 - private
 - protected
 - public
 - default (also called package)
- Often **not used correctly** or understood, especially for programmers who are not well educated in design

Access Control in Java

Specifier	Same class	Same package	Different package subclass	Different package non-subclass
private	Y	n	n	n
package	Y	Y	n	n
protected	Y	Y	Y	n
public	Y	Y	Y	Y

- Most class variables should be private
- Public variables should seldom be used
- Protected variables are particularly dangerous – future programmers can accidentally override (by using the same name) or accidentally use (by mis-typing a similar name)
 - They should be called “unprotected”

Object-Oriented Language Features (Java)

- Method overriding

Allows a method in a subclass to have the same name, arguments and result type as a method in its parent

- Variable hiding

Achieved by defining a variable in a child class that has the same name and type of an inherited variable

- Class constructors

Not inherited in the same way other methods are – must be explicitly called

- Each object has ...

- a declared type : *Parent P;*
- an actual type : *P = new Child ();* or assignment : *P = Pold;*
- Declared and actual types allow uses of the same name to reference different variables with different types

OO Language Feature Terms

- Polymorphic attribute
 - An object reference that can take on various types
 - Type the object reference takes on during execution can change
- Polymorphic method
 - Can accept parameters of different types because it has a parameter that is declared of type Object
- Overloading
 - Using the same name for different constructors or methods in the same class
- Overriding
 - A child class declares an object or method with a name that is already declared in an ancestor class
 - Easily confused with overloading because the two mechanisms have similar names and semantics
 - Overloading is in the same class, overriding is between a class and a descendant

More OO Language Feature Terms

- Members associated with a class are called class or instance variables and methods
 - Static methods can operate only on static variables; not instance variables
 - Instance variables are declared at the class level and are available to objects
- 20 object-oriented mutation operators defined for Java – muJava
- Broken into 4 general categories

OO Mutation Operators—*Encapsulation*

1. AMC — Access Modifier Change

The access level for each instance variable and method is changed to other access levels.

OO Mutation Operators—*Example*

1. *AMC – Access Modifier Change*

point	
	private int x;
Δ1	public int x;
Δ2	protected int x;
Δ3	int x;

OO Mutation Operators—*Inheritance*

2. HVD — *Hiding Variable Deletion*

Each declaration of an overriding or hiding variable is deleted.

3. HVI — *Hiding Variable Insertion*

A declaration is added to hide the declaration of each variable declared in an ancestor.

4. OMD — *Overriding Method Deletion*

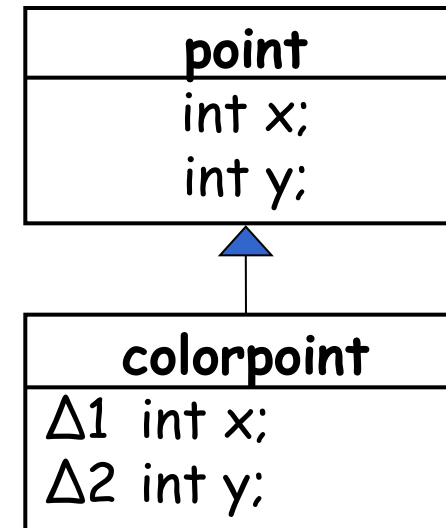
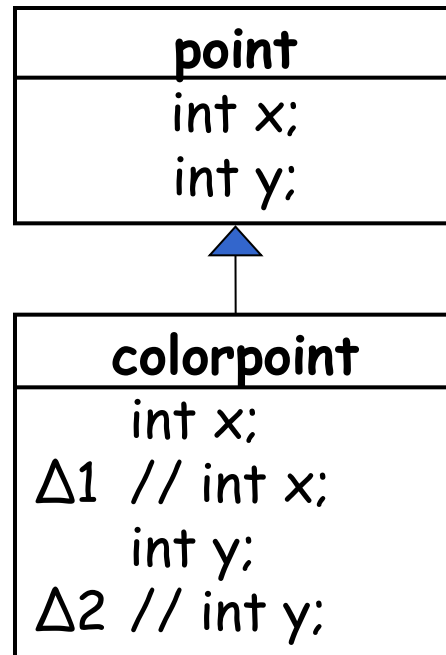
Each entire declaration of an overriding method is deleted.

5. OMM — *Overridden Method Moving*

Each call to an overridden method is moved to the first and last statements of the method and up and down one statement.

OO Mutation Operators—*Example*

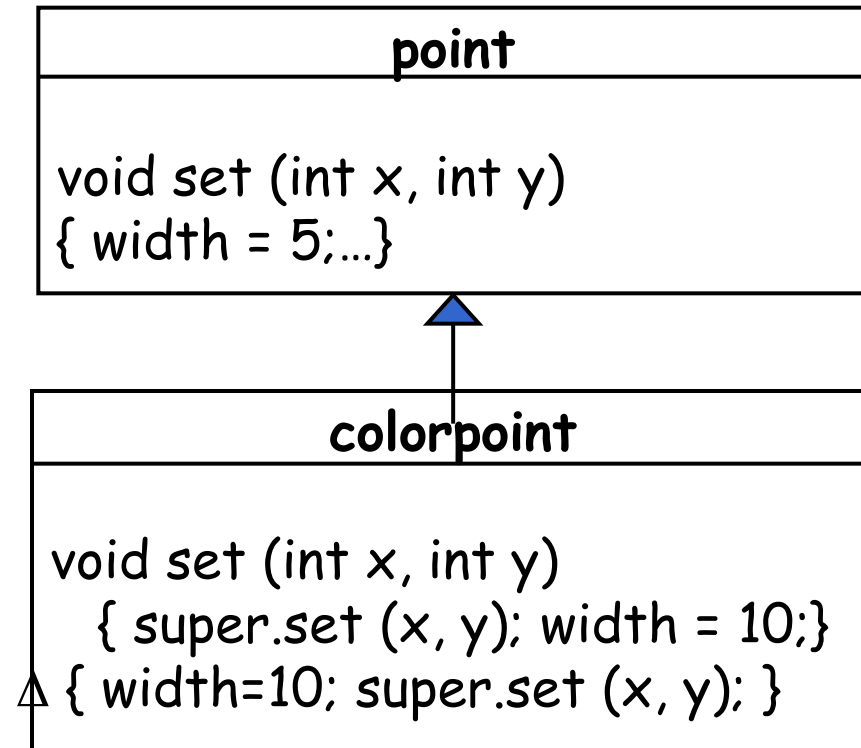
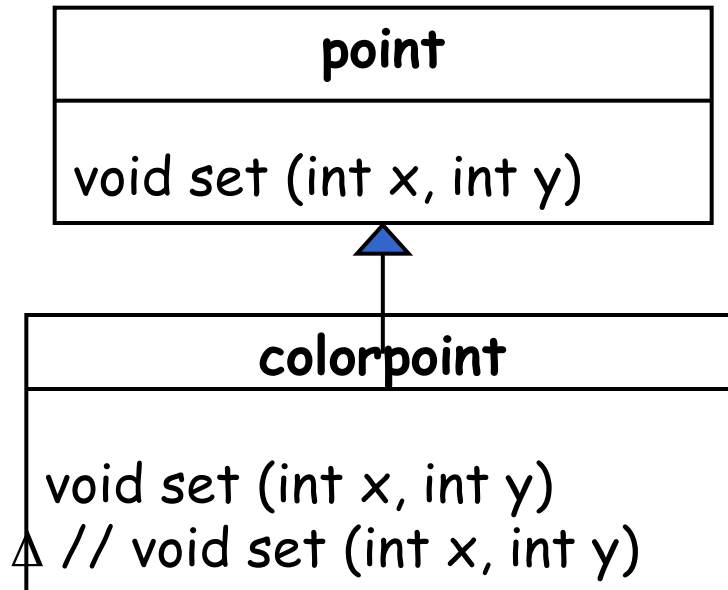
2. *HVD – Hiding Variable Deletion* 3. *HVI – Hiding Variable Insertion*



OO Mutation Operators—*Example*

4. *OMD – Overriding Method Deletion*

5. *OMM – Overriding Method Moving*



OO Mutation Operators—*Inheritance*

6. OMR — *Overridden Method Rename*

Renames the parent's versions of methods that are overridden in a subclass so that the overriding does not affect the parent's method.

7. SKD — *Super Keyword Deletion*

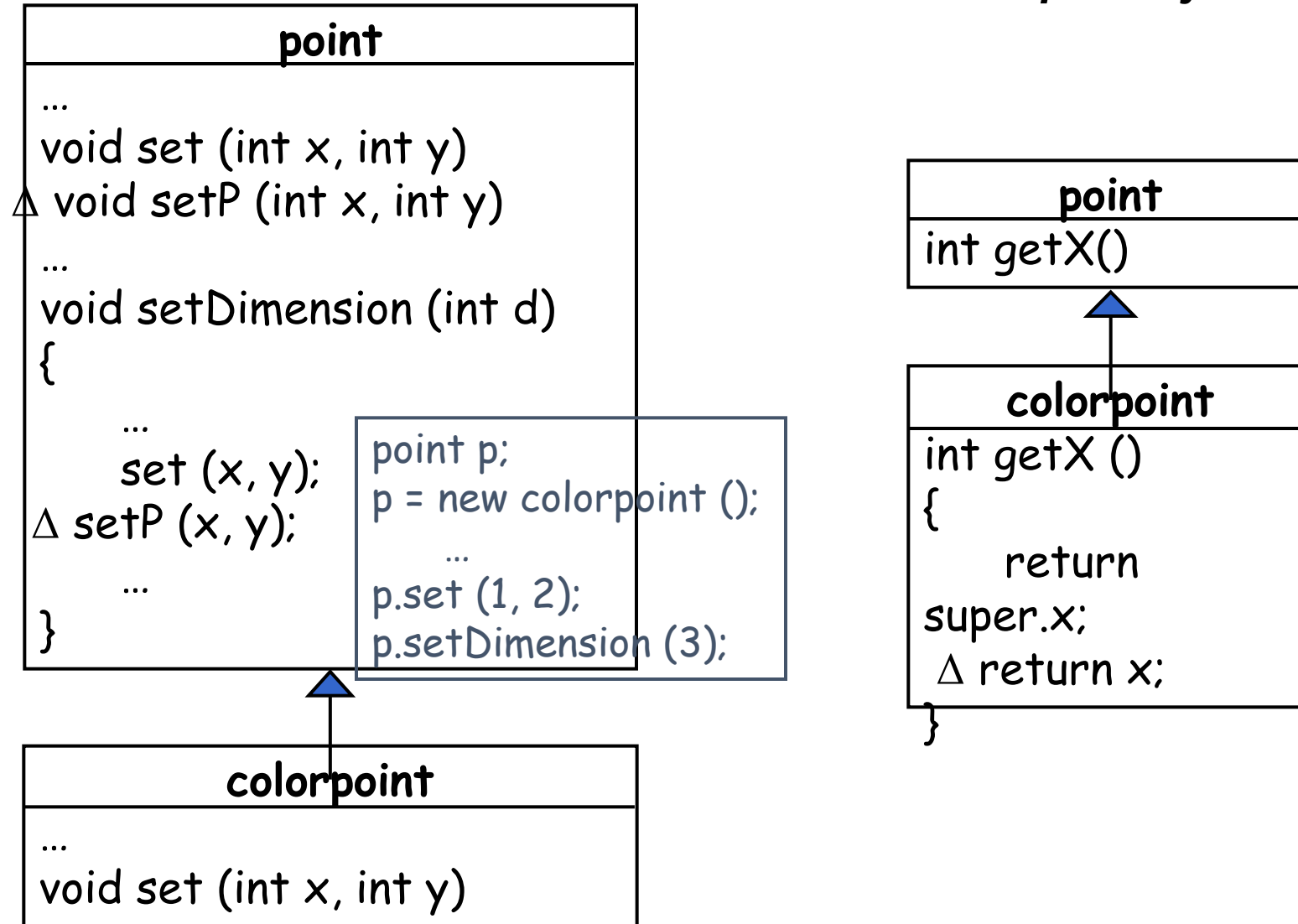
Delete each occurrence of the **super** keyword.

8. PCD — *Parent Constructor Deletion*

Each call to a **super** constructor is deleted.

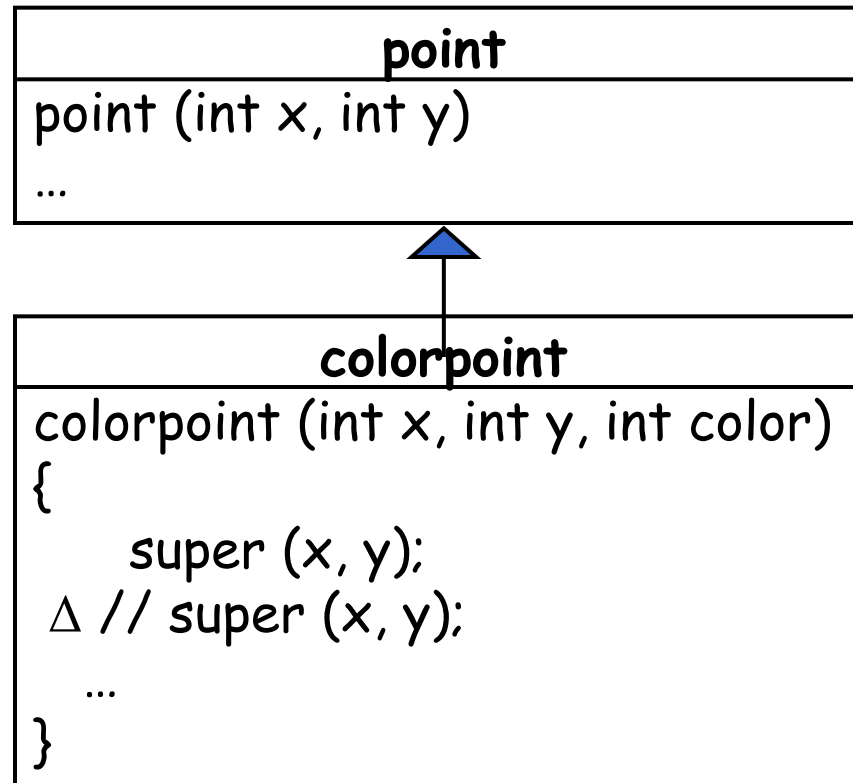
OO Mutation Operators—*Example*

6. *OMR – Overriding Method Rename* 7. *SKD – Super Keyword Deletion*



OO Mutation Operators—*Example*

8. PCD – Parent Constructor Deletion



OO Mutation Operators—*Polymorphism*

9. ATC — *Actual Type Change*

The actual type of a new object is changed in the **new()** statement.

10. DTC — *Declared Type Change*

The declared type of each new object is changed in the declaration.

11. PTC — *Parameter Type Change*

The declared type of each parameter object is changed in the declaration.

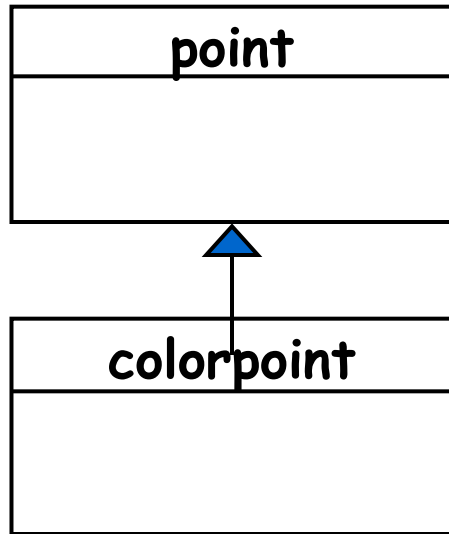
12. RTC — *Reference Type Change*

The right side objects of assignment statements are changed to refer to objects of a compatible type.

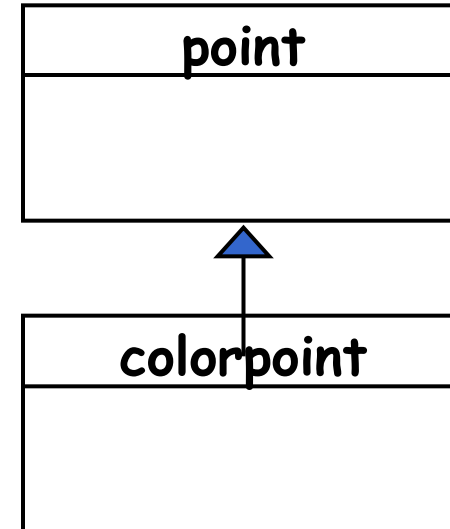
OO Mutation Operators—*Example*

9. *ATC* – *Actual Type Change*

10. *DTC* – *Declared Type Change*



```
point p;
p = new point ();
Δ p = new colorpoint
();
```

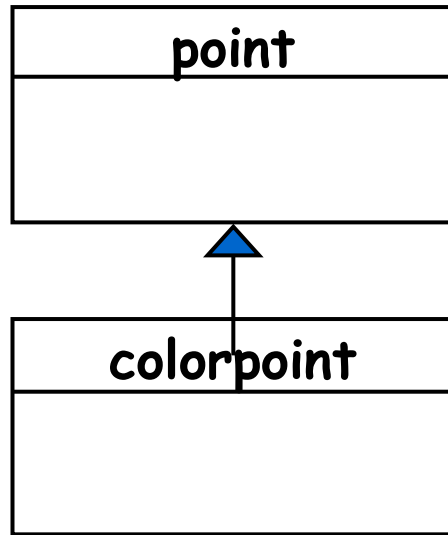


```
point p;
Δ colorpoint p;
p = new colorpoint
();
```

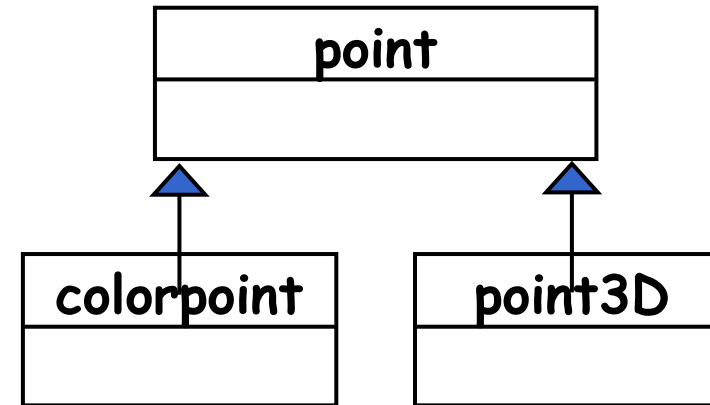
OO Mutation Operators—*Example*

11. PTC – Parameter Type Change

12. RTC – Reference Type Change



```
boolean equals (point p)
{ ... }
Δ boolean equals (colorpoint p)
{ ... }
```



```
point p;
    colorpoint cp = new colorpoint (0,
0);
    point3D p3d = new point3D (0, 0,
0);
Δ p = cp;
Δ p = p3d;
```

OO Mutation Operators—*Polymorphism*

13. OMC — *Overloading Method Change*

For each pair of methods that have the same name, the bodies are interchanged.

14. OMD — *Overloading Method Deletion*

Each overloaded method declaration is deleted, one at a time.

15. AOC — *Argument Order Change*

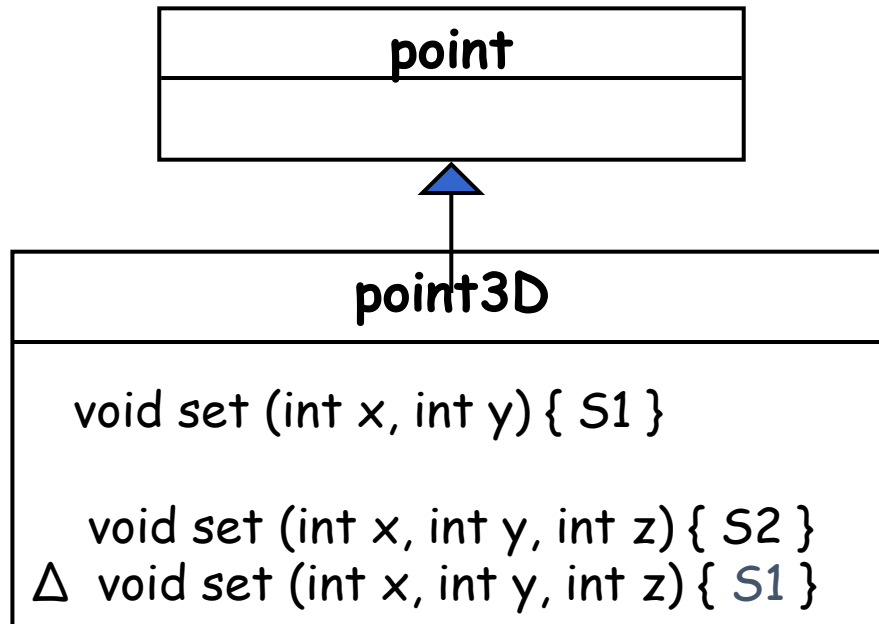
The order of the arguments in method invocations is changed to be the same as that of another overloading method, if one exists.

16. ANC — *Argument Number Change*

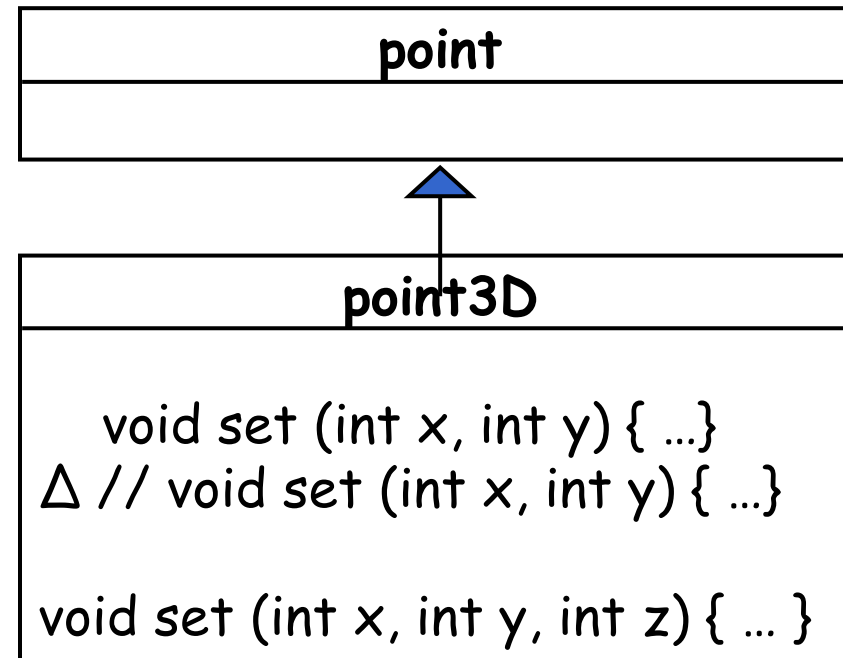
The number of the arguments in method invocations is changed to be the same as that of another overloading method, if one exists.

OO Mutation Operators—*Example*

**13. OMR – Overloading
Method Change**

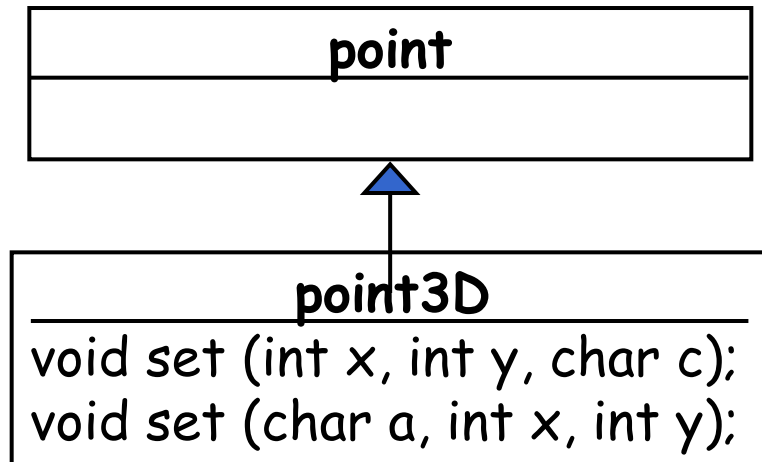


**14. OMD – Overloading
Method Deletion**



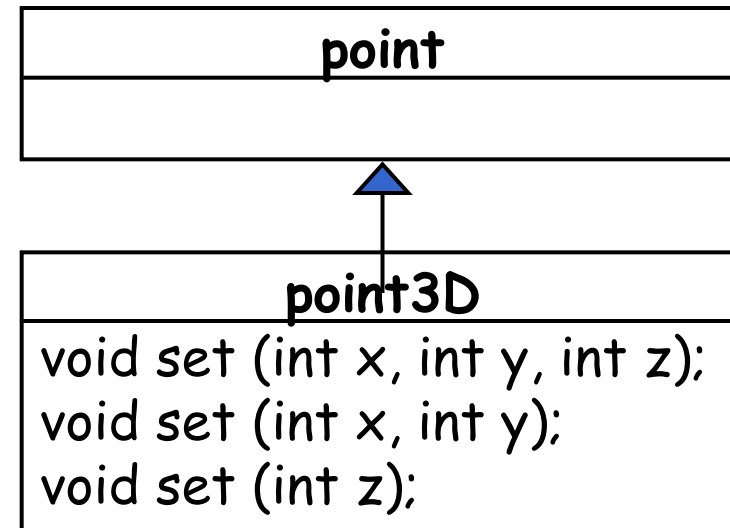
OO Mutation Operators—*Example*

15. AOC – Argument Order Change



```
point3D p;  
p.set (1, 2, 't');  
Δ p.set ('t', 1, 2);
```

16. ANC – Argument Number Change



```
point3D p;  
p.set (1, 2, 3);  
Δ p.set (2, 3);  
Δ p.set (3);
```

OO Mutation Operators—*Language Specific*

17. TKD — *this Keyword Deletion*

Each occurrence of the keyword **this** is deleted.

18. SMC — *Static Modifier Change*

Each instance of the **static** modifier is removed, and the **static** modifier is added to instance variables.

19. VID — *Variable Initialization Deletion*

Remove initialization of each member variable.

20. DCD — *Default Constructor Delete*

Delete each declaration of default constructor (with no parameters).

OO Mutation Operators—*Example*

17. JTD – *This Keyword Deletion*

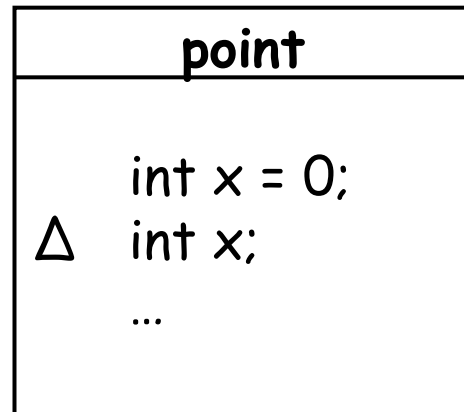
18. JSD – *Static Modifier Change*

point
...
void set (int x, int y)
{
this.x = x;
Δ1 x = x;
this.y = y;
Δ2 y = y;
}
...

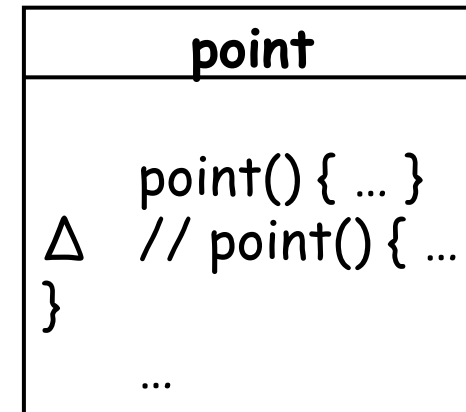
point
private static int x = 0;
Δ1 private int x = 0;
private int y = 0;
Δ2 private static int y = 0;
...

OO Mutation Operators—*Example*

19. VID – Variable Initialization Deletion



20. DCD – Default Constructor Delete



Integration Mutation Summary

- Integration testing often looks at couplings
- Mutation testing modifies callers and callees
- OO mutation focuses on inheritance, polymorphism, dynamic binding, information hiding and overloading
 - The access levels make it easy to make mistakes in OO software
- muJava is an educational / research tool for mutation testing of Java programs
 - <http://cs.gmu.edu/~offutt/mujava/>

Fuzz Testing

- **What is it?** The Open Web [Application Security](#) Project ([OWASP](#)) defines fuzz testing as “a Black Box software testing technique, which basically consists in finding implementation bugs using malformed/semi-malformed data injection in an automated fashion.” Mutation fuzz testing involves simple, random changes to input, such as bit flipping or adding random items to code. Generation fuzz testing, on the other hand, creates new inputs based on specific file formats rather than just adjusting existing ones.
- **What are the limitations?** While it can detect multiple kinds of tactics used by hackers to crash a program, fuzz testing only really works for obvious faults in a Black Box test. Specifying information can be difficult as can obtaining hard data on vulnerability impact. A different TechTarget article advises using fuzz testing in combination with “extensive black box testing, beta testing, and other proven debugging methods.”
- **What are other names for it?** This technique is also known as “fuzzing,” and some of its subtypes include application fuzzing, protocol fuzzing, and file format fuzzing. Mutation fuzzing is sometimes called “dumb fuzzing,” while generative fuzzing is referred to as “intelligent fuzzing.”

fuzz testing == fuzzing

- The original work was inspired by being logged on to a modem during a storm with lots of line noise. And the line noise was generating junk characters that seemingly were causing programs to crash. The noise suggested the term 'fuzz'." --Barton Miller

Types of Fuzzing

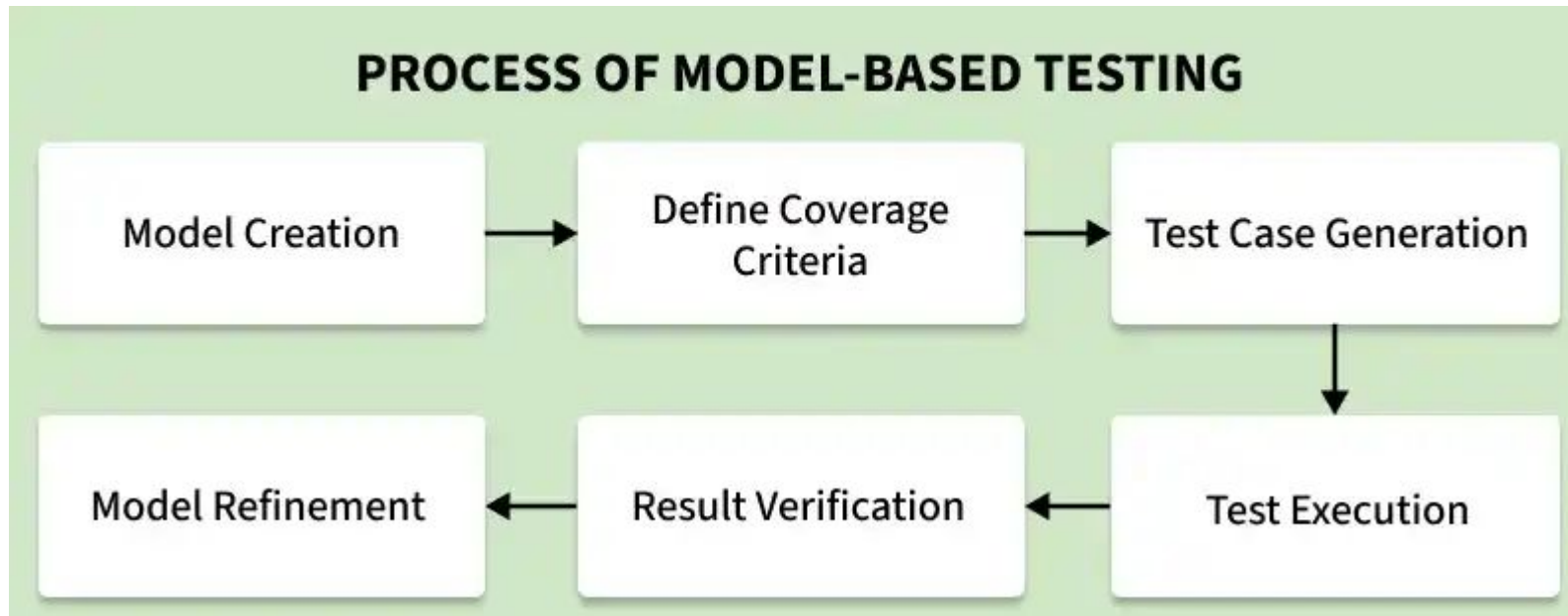
1. Coverage Guided Fuzzing: Focuses on exploring the program's code
 - a. Continuously generates new inputs.
 - b. Uses code coverage to discover new execution paths.
 - c. Particularly interested in crashes, hangs, and abnormal behavior.
 - d. A crash can be reproduced and analyzed to identify the faulty code.
2. Behavioral Fuzzing: Focuses on whether the system behaves correctly.
 - a. Uses specifications or expected behavior.
 - b. Generates unexpected/random inputs.
 - c. Compares actual behavior with expected behavior.
 - d. A difference may indicate a bug or security vulnerability.

Model Based Testing

Model-Based Testing (MBT) is a testing approach where test cases are derived from a model of the system's expected behavior.

Process of MBT

Model-Based Testing (MBT) follows a structured process where a system is modeled, test cases are generated from the model, executed on the system, and results are analyzed to improve quality.



Types of Models Used in MBT

1. FSM / State Transition
2. Statecharts
3. Decision Tables
4. UML Model
5. Markov Models Probabilistic transitions
6. Formal Models

Model-Based Testing Tools

Several tools support model creation and automatic test generation.

- Spec Explorer : Generates tests from models, particularly for .NET applications.
- GraphWalker: Open-source tool for graph-based and state-machine testing.
- Conformiq: Commercial tool for automatic test generation from models and requirements.
- Tricentis Tosca : Model-based/scriptless test automation platform.
- MaTeLo: Uses statistical and Markov-based models for usage-based testing.

Advantages of Model-Based Testing

- Better Test Coverage as explores states, transitions, paths, conditions
- Automated Test Generation
- Early defect detection
- Reusability
- Consistency

Limitations of Model-Based Testing

- Model Creation Cost time, effort, domain knowledge
- Model Complexity
- Model Must Stay Updated
- Tool Dependency

Summary

- If bugs were marbles ...
 - We could get some nice black marbles to judge the quality of test suites
- Since bugs aren't marbles ...
 - Mutation testing rests on some troubling assumptions about seeded faults, which may not be statistically representative of real faults
- Nonetheless ...
 - A model of typical or important faults is invaluable information for designing and assessing test suites

